

Lab. Microcontroladores - Relatório da aula prática 6

Semaforo inteligente com botão para pedido do pedestre

20249035626 - Gabriel de Oliveira Barbosa – Quarta-feira - 2026.1

1. Resumo da prática:

Nesta prática, foi dada a instrução pelo professor da disciplina para construirmos um sistema em Arduino *NANO* que realizasse a função de semáforo inteligente com pedido de pedestre por meio do botão *push*, o objetivo deste botão é incluir no ciclo de funcionamento dos leds do semáforo a função de passagem do pedestre por meio do semáforo dos pedestres. O código visto em 6.1 mostra como as cláusulas comparativas interagem entre si, sob os tempos definidos para funcionar corretamente.

2. Objetivos:

Adquirir experiência no campo de montagem de sistemas embarcados com uso do Arduino, e obter uma maior noção do uso de códigos para executar funções conjuntas mais complexas, como neste caso a do semáforo inteligente com botão *push* com resistor interno.

3. Material Utilizado:

- 3.1 |1x Arduino NANO.
- 3.2 |Cabo Micro-USB.
- 3.3 |1x Protoboards.
- 3.4 |8x Cabos Jumper macho-macho.
- 3.5 |2x Led verde.
- 3.6 |2x Led vermelho.
- 3.7 |1x Led amarelo.
- 3.7 |1x Botão Push.
- 3.8 |Computador/Notebook.
- 3.9 |2x Resistor 220ohms.

4. Montagens:

Similar a estrutura de funcionamento das montagens que envolviam o uso de leds, este projeto envolve agora o uso de 2 (dois) pares de leds verdes e vermelhos, e um amarelo, que são postos em posições na placa de ensaio que mantenham a ideia de que são dois semáforos distintos, de pedestre e o veicular, sendo o primeiro dividido entre os leds verde e vermelho (onde o de cor verde pisca para mudar de fase), e no semáforo veicular tem o led amarelo para indicar o sinal de alerta entre as mudanças de estado do avanço, e da parada dos veículos. Deverá ser posicionado na protoboard o Arduino *NANO*, e posicionado as alimentações de corrente e o aterramento nos pinos definidos no corpo da placa, a seguir devem ser usados para a alimentação dos leds (que se conectaram aos pinos digitais) os resistores de 220ohms para evitar a queima dos díodos, e o aterramento apropriado na outra extremidade. Quanto a instalação do botão *push*, deve ser feita por meio da alimentação (em um pino digital) e a outra extremidade paralela do mesmo lado o aterramento. Após esses passos estarem completos para a montagem, deverá ser posto no Nano o cabo Micro-Usb para alimentação e comunicação, e rodado o código por meio de uma IDE (sendo essa a Arduino IDE, visível na figura 2), depurado o código no terminal do aplicativo, e enviado para o sistema embarcado. O processador utilizado neste projeto foi o ATmega328P, e verificado com a porta respectiva onde foi posta a saída USB A do cabo Micro-Usb, tudo foi usado em um computador pessoal externo. O resultado da montagem já feita pode ser vista na figura 4, no tópico 7, nela há a inclusão do *buzzer*, embora não utilizado neste atual

projeto, ele não interfere em nada no funcionamento deste sistema nem no código.

5. Resultados da montagem:

Houve êxito na execução do projeto, após revisado o código visto em 6.1, o sistema retornou à função de inclusão do ciclo de pedido do semáforo do pedestre. Originalmente haviam alguns problemas entre os segmentos comparativos e entre os casos de Switch, além de outros empecilhos para quando o envio do código para o sistema embarcado, por conta de incompatibilidade dos modelos de processador e por um erro de falha da IDE por conta da duplicação das instâncias.

6. Bloco de Código:

6.1- Abaixo está escrito o código utilizado para a montagem da prática 6.

```
const int PINO_VERDE_V=10;
const int PINO_AMARELO_V=11;
const int PINO_VERMELHO_V=12;
const int PINO_VERDE_P=7;
const int PINO_VERMELHO_P=8;
const int PINO_BOTAO_P=2;

const unsigned long TEMPO_VERDE_V=8000;
const unsigned long
TEMPO_PISCA_VERDE_V=2000;
const unsigned long
TEMPO_AMARELO_V=3000;
const unsigned long
TEMPO_VERMELHO_V_MIN=5000;
const unsigned long TEMPO_VERDE_P=7000;
const unsigned long
TEMPO_VERMELHO_P_PISCANTE=3000;
const unsigned long
TEMPO_PISCA_INTERVALO=300;

enum EstadoSemaforo{
    VEICULAR_VERDE,
    VEICULAR_VERDE_PISCANDO,
    VEICULAR_AMARELO,
    VEICULAR_VERMELHO,
```

```
    PEDESTRE_PREPARA,
    PEDESTRE_VERDE,
    PEDESTRE_VERMELHO_PISCANDO,
    PEDESTRE_FINALIZA
};
```

```
EstadoSemaforo
```

```
estadoAtual=VEICULAR_VERDE;
```

```
unsigned long tempoEntradaEstado=0;
```

```
unsigned long ultimoTempoPisca=0;
```

```
bool ledPiscaEstado=LOW;
```

```
bool botaoPedestrePressionado=false;
```

```
bool pedidoPedestrePendente=false;
```

```
int estadoBotaoEstavel=HIGH;
```

```
int ultimoEstadoLeitura=HIGH;
```

```
unsigned long ultimoDebounceTempo=0;
```

```
const unsigned long debounceDelay=50;
```

```
void setup(){
```

```
    pinMode(PINO_VERDE_V,OUTPUT);
```

```
    pinMode(PINO_AMARELO_V,OUTPUT);
```

```
    pinMode(PINO_VERMELHO_V,OUTPUT);
```

```
    pinMode(PINO_VERDE_P,OUTPUT);
```

```
    pinMode(PINO_VERMELHO_P,OUTPUT);
```

```
    pinMode(PINO_BOTAO_P,INPUT_PULLUP);
```

```
    Serial.begin(9600);
```

```
    Serial.println("Semaforo com Chamada de  
Pedestre Iniciado.");
```

```
    mudarParaEstado(VEICULAR_VERDE);
```

```
}
```

```
void loop(){
```

```
    verificarBotaoPedestre();
```

```
    controlarSemaforo();
```

```
}
```

```
void verificarBotaoPedestre(){
```

```
int leituraBotao=digitalRead(PINO_BOTAO_P);
```

```
if(leituraBotao!=ultimoEstadoLeitura)
```

```
ultimoDebounceTempo=millis();
```

```
if((millis()-
```

```
ultimoDebounceTempo)>debounceDelay){
```

```
if(leituraBotao!=estadoBotaoEstavel){
```

```
estadoBotaoEstavel=leituraBotao;
```

```
if(estadoBotaoEstavel==LOW&&!pedidoPedestr  
ePendente){
```

```
botaoPedestrePressionado=true;
```

```
pedidoPedestrePendente=true;
```

```
Serial.println("Botao de pedestre pressionado!");
```

```
}
```

```

}
}
ultimoEstadoLeitura=leituraBotao;
}

void controlarSemaforo(){
unsigned long tempoNoEstadoAtual=millis()-
tempoEntradaEstado;

switch(estadoAtual){
case VEICULAR_VERDE:
if(tempoNoEstadoAtual>=(TEMPO_VERDE_V-
TEMPO_PISCA_VERDE_V)){
if(pedidoPedestrePendente)
mudarParaEstado(VEICULAR_VERDE_PISCA
NDO);
else
if(tempoNoEstadoAtual>=TEMPO_VERDE_V)

mudarParaEstado(VEICULAR_VERDE_PISCA
NDO);
}
break;

case VEICULAR_VERDE_PISCANDO:
piscarLed(PINO_VERDE_V,TEMPO_PISCA_VE
RDE_V);
if(tempoNoEstadoAtual>=TEMPO_PISCA_VER
DE_V)
    mudarParaEstado(VEICULAR_AMARELO);
break;

case VEICULAR_AMARELO:
if(tempoNoEstadoAtual>=TEMPO_AMARELO_
V)
mudarParaEstado(VEICULAR_VERMELHO);
break;

case VEICULAR_VERMELHO:
if(pedidoPedestrePendente&&tempoNoEstadoAt
ual>=TEMPO_VERMELHO_V_MIN/2)
mudarParaEstado(PEDESTRE_PREPARA);
else
if(!pedidoPedestrePendente&&tempoNoEstadoA
tual>=TEMPO_VERMELHO_V_MIN)

mudarParaEstado(VEICULAR_VERDE);
break;

case PEDESTRE_PREPARA:
if(tempoNoEstadoAtual>=500)
mudarParaEstado(PEDESTRE_VERDE);

```

```

break;

case PEDESTRE_VERDE:
if(tempoNoEstadoAtual>=TEMPO_VERDE_P)
mudarParaEstado(PEDESTRE_VERMELHO_P_I
SCANDO);
break;

case PEDESTRE_VERMELHO_PISCANDO:
piscarLed(PINO_VERMELHO_P,TEMPO_VER
MELHO_P_PISCANTE);
if(tempoNoEstadoAtual>=TEMPO_VERMELHO
_P_PISCANTE)
mudarParaEstado(PEDESTRE_FINALIZA);
break;

case PEDESTRE_FINALIZA:
if(tempoNoEstadoAtual>=1000){
pedidoPedestrePendente=false;
botaoPedestrePressionado=false;
mudarParaEstado(VEICULAR_VERDE);
}
break;
}
}

void mudarParaEstado(EstadoSemaforo
novoEstado){
estadoAtual=novoEstado;
tempoEntradaEstado=millis();
ledPiscaEstado=LOW;

apagarTodosLedsVeiculares();
apagarTodosLedsPedestre();
Serial.print("Mudando para estado: ");

switch(estadoAtual){
case VEICULAR_VERDE:
Serial.println("VEICULAR_VERDE");
digitalWrite(PINO_VERDE_V,HIGH);
digitalWrite(PINO_VERMELHO_P,HIGH);
break;

case VEICULAR_VERDE_PISCANDO:
Serial.println("VEICULAR_VERDE_PISCANDO"
);
digitalWrite(PINO_VERMELHO_P,HIGH);
break;

case VEICULAR_AMARELO:
Serial.println("VEICULAR_AMARELO");
digitalWrite(PINO_AMARELO_V,HIGH);
digitalWrite(PINO_VERMELHO_P,HIGH);

```

```

break;

case VEICULAR_VERMELHO:
Serial.println("VEICULAR_VERMELHO");
digitalWrite(PINO_VERMELHO_V,HIGH);
digitalWrite(PINO_VERMELHO_P,HIGH);
break;

case PEDESTRE_PREPARA:
Serial.println("PEDESTRE_PREPARA");
digitalWrite(PINO_VERMELHO_V,HIGH);
digitalWrite(PINO_VERMELHO_P,HIGH);
break;

case PEDESTRE_VERDE:
Serial.println("PEDESTRE_VERDE");
digitalWrite(PINO_VERMELHO_V,HIGH);
digitalWrite(PINO_VERDE_P,HIGH);
break;

case PEDESTRE_VERMELHO_PISCANDO:
Serial.println("PEDESTRE_VERMELHO_PISCA
NDO");

digitalWrite(PINO_VERMELHO_V,HIGH);
break;

case PEDESTRE_FINALIZA:
Serial.println("PEDESTRE_FINALIZA");
digitalWrite(PINO_VERMELHO_V,HIGH);
digitalWrite(PINO_VERMELHO_P,HIGH);
    break;
}
}
void piscarLed(int pinoLed,unsigned long
duracaoTotalPisca){

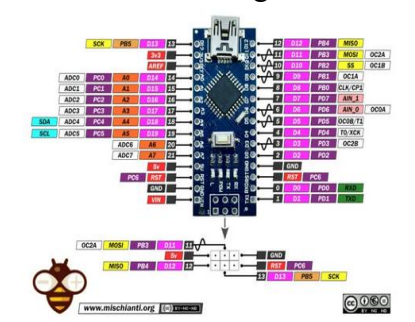
if((millis()-
ultimoTempoPisca)>TEMPO_PISCA_INTERVA
LO){
    ultimoTempoPisca=millis();
    ledPiscaEstado=!ledPiscaEstado;
    digitalWrite(pinoLed,ledPiscaEstado);
}
}
void apagarTodosLedsVeiculares(){
digitalWrite(PINO_VERDE_V,LOW);
digitalWrite(PINO_AMARELO_V,LOW);
digitalWrite(PINO_VERMELHO_V,LOW);
}
void apagarTodosLedsPedestre(){
digitalWrite(PINO_VERDE_P,LOW);
digitalWrite(PINO_VERMELHO_P,LOW);
}
}

```

7. Referencias e Imagens:

- Arduino NANO, informações, acesso em: <https://mischianti.org/arduino-nano-high-resolution-pinout-datasheet-and-specs>
- Arduino IDE, software utilizado.
- Documento sobre a prática 6, disponível no SIGAA.
- Datasheet do botão *push*, acesso em: <https://components101.com/switches/push-button>
- Datasheet do LED, acesso em: <https://components101.com/diodes/5mm-round-led>

Arduino NANO, figura 1:



Interface do Arduino IDE, figura 2:

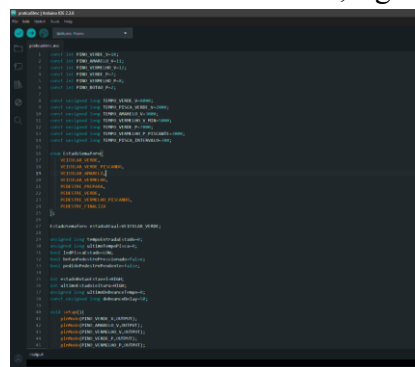


Diagrama do Led utilizado, figura 3:

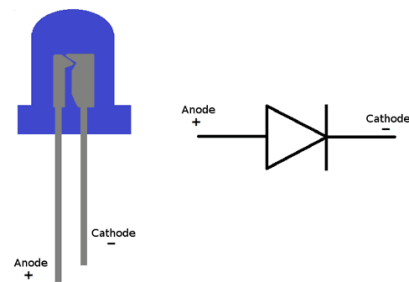
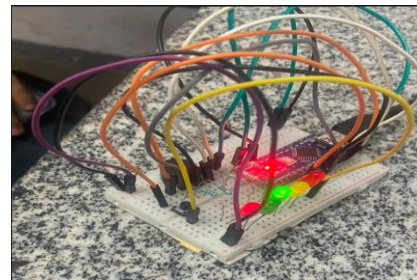


Imagem da montagem concluída, figura 4:



Obs: Embora a montagem tenha a inclusão de um buzzer sonoro, ela não interfere no funcionamento da montagem descrita no tópico 3, nem no código do tópico 6. Esse componente elétrico será mais bem trabalhado na prática 7.